



MODUL 14 · SEO, METADATA & PERFORMANCE

Core Web Vitals & React Compiler (Auto- Memoization)

Tiga metrik Google yang menentukan ranking. React Compiler yang eliminasi useMemo manual.



Core Web Vitals 2024+ — Tiga Metrik Utama

LCP, CLS, INP (menggantikan FID sejak Maret 2024). Target nilai untuk 'Good'.

LCP

Largest Contentful Paint

Good: ≤ 2.5 detik

Waktu hingga konten terbesar (gambar/teks hero) muncul di layar.

1. next/image dengan priority={true} untuk hero
2. Server Component untuk tidak ada loading flash
3. CDN untuk konten statis

CLS

Cumulative Layout Shift

Good: ≤ 0.1

Seberapa banyak halaman bergeser saat loading (layout shift).

1. next/image dengan width+height (reserve space)
2. next/font (zero FOUT)
3. Skeleton loader dengan ukuran yang sama

INP

Interaction to Next Paint

Good: ≤ 200 ms

Responsivitas terhadap interaksi user: klik, ketik, tap. (Gantikan FID 2024)

1. startTransition untuk update non-urgent
2. React Compiler (auto-memoization)
3. Avoid expensive computation di event handler

React Compiler — Auto-Memoization

Enable di Next.js 16. Compiler otomatis tambah memoization — kurangi manual useMemo/useCallback.

● ● ● Aktifkan React Compiler

```
// next.config.ts — aktifkan React Compiler
import type { NextConfig } from "next"

const nextConfig: NextConfig = {
  experimental: {
    reactCompiler: true, // ← aktifkan!
  },
}
export default nextConfig

// Apa yang dilakukan React Compiler?
// Otomatis memoize:
// - Komponen yang tidak berubah (memo)
// - Nilai computed yang tidak berubah (useMemo)
// - Fungsi yang tidak berubah (useCallback)

// Sebelum compiler:
function PostList({ posts, onLike }) {
  const sorted = useMemo( // manual!
    () => posts.sort(...), [posts]
  )
  const handleLike = useCallback( // manual!
    (id) => onLike(id), [onLike]
  )
  return <ul>{sorted.map(...)}</ul>
}

// Setelah compiler: compiler otomatis add memo!
function PostList({ posts, onLike }) {
  const sorted = posts.sort(...) // compiler memoize ini
  return <ul>{sorted.map(...)}</ul>
}
```

Kapan Masih Butuh Manual?

React Compiler TIDAK 100% replace manual hooks.

Masih butuh useMemo/useCallback:

- Saat compiler tidak bisa prove bahwa value aman untuk memoize
- Explicit opt-out dengan compiler directive
- Cross-component state yang complex

Masih butuh memo():

- Untuk komponen yang ada di kritikal path performance

Gunakan:

use no memo

// directive untuk opt-out component

// dari compiler memoization

React Compiler = tool bantu, bukan silver bullet.

Performance Checklist — Ship yang Cepat

Checklist praktis sebelum deploy ke production. Audit ini di setiap release.

Images

- ✓ Semua gambar pakai `<Image>` dari `next/image`
- ✓ `sizes` prop di semua responsive image
- ✓ `priority={true}` HANYA untuk hero/above-fold

Fonts

- ✓ Semua font pakai `next/font` (bukan CDN)
- ✓ Subset minimal (latin saja jika cukup)
- ✓ Maksimal 2 font family

JavaScript

- ✓ `'use client'` hanya jika benar-benar perlu
- ✓ React Compiler diaktifkan
- ✓ Dynamic import untuk komponen besar

Caching

- ✓ `'use cache'` untuk data yang bisa stale
- ✓ `revalidateTag` setelah setiap mutasi
- ✓ Static pages di-prerender saat build

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — INP (Interaction to Next Paint) menggantikan metrik Core Web Vitals apa?

A) CLS (Cumulative Layout Shift)

B) FID (First Input Delay) — sejak Maret 2024

C) LCP (Largest Contentful Paint)

Q2 — Apa yang dilakukan React Compiler di Next.js 16?

A) Compile TypeScript lebih cepat

B) Otomatis menambahkan memoization
(useMemo/useCallback/memo) tanpa harus
manual

C) Compile JSX ke WebAssembly

Q3 — Mengapa 'use client' harus digunakan sesedikit mungkin untuk performance?

A) 'use client' menyebabkan error

B) Client Components bundle javascript ke
browser — lebih banyak Client Component =
bundle lebih besar

C) Server Components tidak support hooks

Tugas Mandiri

Audit dan optimalkan performance project sebelum deploy.

01 React Compiler

Aktifkan `experimental.reactCompiler` di `next.config.ts`. Hapus manual `useMemo` dan `useCallback` yang tidak diperlukan. Cek tidak ada regression.

02 Lighthouse Audit

`bun run build && bun run start` → Lighthouse audit. Target: Performance ≥ 90 , SEO ≥ 95 . Catat skor sebelum dan sesudah optimasi.

03 Bundle Analysis

`bun add @next/bundle-analyzer`. Aktifkan di `next.config.ts`. Jalankan `ANALYZE=true bun run build`. Identifikasi bundle terbesar.

04 CWV Test

Setelah deploy ke Vercel: buka PageSpeed Insights (pagespeed.web.dev). Masukkan URL. Target: LCP $\leq 2.5s$, CLS ≤ 0.1 , INP $\leq 200ms$.

💡 Tools: Lighthouse (DevTools), PageSpeed Insights, web.dev/measure. Run sebelum dan sesudah optimasi.

Yang Sudah Kita Pelajari

- ✓ Core Web Vitals: LCP ($\leq 2.5s$), CLS (≤ 0.1), INP ($\leq 200ms$). INP menggantikan FID 2024.
- ✓ LCP: next/image priority. CLS: next/image size + next/font. INP: startTransition + Compiler.
- ✓ React Compiler: aktifkan di experimental.reactCompiler. Auto-memoize tanpa manual hooks.
- ✓ Performance checklist: Images, Fonts, JavaScript, Caching — audit setiap release.
- ✓ Tools: Lighthouse, PageSpeed Insights, bundle-analyzer. Target Performance ≥ 90 .

→ Selanjutnya: **Modul 15 — Deployment & Production**

